고객을 세그먼테이션하자 [프로젝트] 이미현

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM sonic-proxy-473602-k2.project01.data
LIMIT 10
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIGH...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT ...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- **Q. 총 몇개의 컬럼으로 구성된 데이터 셋인가요?**

  8개

  1. InvoiceNo
  2. StockCode
  3. Description
  4. Quantity
  5. InvoiceDate
  6. UnitPrice
  7. CustomerID
  8. Country

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS total_rows
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	total_rows
1	541909

> 📌 데이터 업로드와 처리 시간 단축을 위해
> (1) 내가 우선 원하는 결과가 잘 나오는 쿼리인지 확인한 후
> (2) LIMIT을 제거하여
> (3) 전체 데이터셋에 적용

데이터 수 세기

- **Q. Quantity 컬럼은 어떤 데이터 타입의 데이터들이 저장되어 있나요?**

  STRING 문자열
  INTEGER 정수
  TIMESTAMP* 시간
  FLOAT 실수

*TIME의 특정 시점에 STAMP를 붙인 것. 시작 지점을 주로 UTC를 사용
*마이크로초 단위 정밀도를 갖는 절대 시점

sonic-proxy-473602-k2 / ··· / Tables / data 쿼리 다음에서 열기 공유 복사 스냅샷 삭제 내보내기 새로고침

스키마 세부정보 미리보기 테이블 탐색기 프리뷰 통계 계보 데이터 프로필 데이터 품질

필터 속성 이름 또는 값 입력

필드 이름	유형	모드	설명	키	대조	기본값	정책 태그	데이터 정책
InvoiceNo	STRING	NULLABLE	-	-	-	-	-	-
StockCode	STRING	NULLABLE	-	-	-	-	-	-
Description	STRING	NULLABLE	-	-	-	-	-	-
Quantity	INTEGER	NULLABLE	-	-	-	-	-	-
InvoiceDate	TIMESTAMP	NULLABLE	-	-	-	-	-	-
UnitPrice	FLOAT	NULLABLE	-	-	-	-	-	-
CustomerID	INTEGER	NULLABLE	-	-	-	-	-	-
Country	STRING	NULLABLE	-	-	-	-	-	-

- **Q. UnitPrice 컬럼은 어떤 타입의 데이터들이 저장되어 있나요?**

  FLOAT : 부동 소수점, 실수

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT
  COUNT(InvoiceNo)   AS COUNT_InvoiceNo,
  COUNT(StockCode)   AS COUNT_StockCode,
  COUNT(Description) AS COUNT_Description,
  COUNT(Quantity)    AS COUNT_Quantity,
  COUNT(InvoiceDate) AS COUNT_InvoiceDate,
  COUNT(UnitPrice)   AS COUNT_UnitPrice,
  COUNT(CustomerID)  AS COUNT_CustomerID,
  COUNT(Country)     AS COUNT_Country
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Descripti...	COUNT_Quantity	COUNT_InvoiceD...	COUNT_UnitPrice	COUNT_Custome...	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

Q. 누락된 값이 있는 모든 컬럼은 무엇인가요?

총 541,909행이 아닌 컬럼은
1. Description
2. CustomerID

> **1. 결측치(Missing Values)**
> - 데이터 집합에서 값이 누락되거나 기록되지 않은 경우 = 비어있는 값
> - 'NaN' (Not-a-Number) 또는 'null'로 표시
>
> - 결측치는 데이터 수집 과정에서 발생, 실제 값이 존재하지 않거나 기록되지 않은 경우에 나타남
> - 결측치 처리 방법 : 삭제, 대체, 예측 등 → 데이터 분석 목적에 따라 처리 방법 결정
>
> **2. 이상치(Outliers)**
> - 데이터 집합에서 다른 관측치들과 크게 다른 값을 가지는 데이터
> - 이상치는 오류, 측정 오류, 드문 사건 등으로 인해 발생
> - 데이터의 분석 및 모델링에 부정적인 영향을 미칠 수 있음
> - 이상치 탐지 방법 : 통계적 방법, 시각화 도구, 머신러닝 알고리즘 등

> 데이터 전처리 방법
> 1. 결측치 처리
> - 비어 있는 값
> - 대표적인 처리 방법 : 삭제, 다른 값들의 평균/중앙값/최빈값으로 대체
>
> 2. 중복값 처리
> - 동일한 정보를 가진 반복된 행

> - 단, 경우에 따라 특정 조건이나 규칙에 따라 중복 데이터 유지 필요성이 있으니 주의
>
> 3. 정규화 및 표준화
> - 데이터 범위를 일정하게 조정, 데이터의 스케일을 변경
> - 다양한 방법이 존재
> - 예, 데이터 값 차이가 클 경우 데이터 범위를 0~1 사이의 숫자 형태로 스케일링 진행 or 로그 변환 or 제곱근 변환
>
> 4. 범주형(명목형) 데이터의 인코딩
> - 성별, 구매 지역 등의 범주형(명목형) 데이터 → 수치형으로 변환하는 전처리 방법
> - 예, 남 0 여 1
>
> 5. 이상치 분석 및 처리
> - 통계적인 방법 or 시각적 도구 활용 → 아웃라이어 찾아내기
>
> ⇒ 데이터 전처리 방법에 정답은 없음
> ⇒ 데이터의 특성과 분포를 보면서 적절한 전처리 방법을 적용해야 함

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data

UNION ALL

SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data

UNION ALL

SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data

UNION ALL

SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data

UNION ALL

SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data

UNION ALL

SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data
```

```
UNION ALL

SELECT
   'CustomerID' AS column_name,
   ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data

UNION ALL

SELECT
   'Country' AS column_name,
   ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	column_name	missing_percenta...
1	Country	0.0
2	InvoiceDate	0.0
3	InvoiceNo	0.0
4	CustomerID	24.93
5	StockCode	0.0
6	Quantity	0.0
7	Description	0.27
8	UnitPrice	0.0

→ 결측치 비율
- CustomerID : 24.93% → 데이터 편향이 큰 편(약 1/4 정도임) ⇒ 누락된 행 제거
- Description : 0.27% → 데이터 편향이 적은 편

> ✅ 데이터 편향
> 데이터가 특정 경향을 가지고 있어 전체적인 분석이나 결과가 왜곡되는 현상
>
> ✅ 참고: 데이터 노이즈
> 데이터에 관련 없는 정보가 포함되어 있어 데이터의 질이 저하되고 분석의 정확도가 떨어지는 것

결측치 처리 전략

- **`StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기**

```
SELECT DISTINCT StockCode, Description
FROM sonic-proxy-473602-k2.project01.data
WHERE StockCode = '85123A';
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	StockCode	Description
1	85123A	WHITE HANGING HEART T-LIGHT HOLDER
2	85123A	?
3	85123A	wrongly marked carton 22804
4	85123A	CREAM HANGING HEART T-LIGHT HOLDER

→ 같은 StockCode인데 Description이 모두 다름 = 일관되지 않은 기록 오류

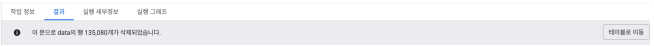
→ 다른 설명으로 대체하는 것은 데이터 신뢰도가 떨어지며 & 결측치 비율이 0.27%로 낮기에

⇒ 해당 행을 삭제하는 것으로 결정해도 무방

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM sonic-proxy-473602-k2.project01.data
-- CustomerID 혹은 Description이 NULL일 경우 데이터 삭제
WHERE CustomerID IS NULL
  OR Description IS NULL;
```



→ 총 135,080개의 행 삭제 완료 = 노드 결과와 동일 확인 완료!

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
-- 1. 중복 그룹 수를 계산하는 최종 쿼리
SELECT
   COUNT(*) AS duplicate_groups  -- HAVING 조건을 만족하는 중복 그룹의 개수 계산
FROM (
   -- 2. 8개 컬럼을 기준으로 그룹핑하여 각 그룹의 행 개수를 계산
   SELECT
      InvoiceNo,
      StockCode,
      Description,
      Quantity,
      InvoiceDate,
      UnitPrice,
      CustomerID,
      Country,
      COUNT(*) AS cnt  -- 각 그룹 내 행 개수를 cnt라는 이름으로 저장
   FROM sonic-proxy-473602-k2.project01.data
   GROUP BY
      InvoiceNo,
      StockCode,
      Description,
      Quantity,
      InvoiceDate,
      UnitPrice,
      CustomerID,
      Country
   HAVING COUNT(*) > 1  -- 행이 2개 이상인 그룹(=중복 그룹)만 선택
) duplicates;  -- 서브쿼리 규칙에 의해 별칭, COUNT(*) 계산 시 사용
```

Q. 중복된 값이 있는 행의 개수는 몇 개인가요?

4,837행

중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.data AS
SELECT DISTINCT *
FROM sonic-proxy-473602-k2.project01.data;
```

```
-- 1. DISTINCT 적용 후 남은 행 수 확인
-- SELECT COUNT(*) AS remaining_rows
-- FROM (
--    SELECT DISTINCT *
--    FROM sonic-proxy-473602-k2.project01.data
-- ) duplicates_removed;
-- 결과 : 401,604행
```

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 이름이 data인 테이블이 교체되었습니다. 테이블로 이동

중복값 처리 후 남은 데이터 행의 개수 확인

```
SELECT COUNT(*) AS total_rows
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	total_rows
1	401604

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- **고유(unique)한 InvoiceNo 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo) AS total_rows
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	total_rows
1	22190

Q. 유니크한 InvoiceNo 는 몇 개인가요?

22,190개

- **고유한 InvoiceNo 를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM sonic-proxy-473602-k2.project01.data
LIMIT 100;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318

*100개 모두 출력 완료 / 이미지 캡쳐는 10개에서 자름

→ 특이 사항 : InvoiceNo에 숫자가 아니라, C로 시작하는 것들 존재

⇒ **InvoiceNo 가 'C'로 시작한다면 취소한 거래**

- `InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM sonic-proxy-473602-k2.project01.data
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
6	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
7	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
8	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
9	C547388	22645	CERAMIC HEART FAIRY CAKE M...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway
10	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
11	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
12	C549955	22839	3 TIER CAKE TIN GREEN AND C...	-2	2011-04-13 13:38:00 UTC	14.95	12359	Cyprus
13	C549955	22666	RECIPE BOX PANTRY YELLOW D...	-2	2011-04-13 13:38:00 UTC	2.95	12359	Cyprus
14	C580165	22826	LOVE SEAT ANTIQUE WHITE ME...	-1	2011-12-02 11:21:00 UTC	42.5	12359	Cyprus
15	C580165	23245	SET OF 3 REGENCY CAKE TINS	-2	2011-12-02 11:21:00 UTC	4.95	12359	Cyprus
16	C580165	22797	CHEST OF DRAWERS GINGHAM...	-2	2011-12-02 11:21:00 UTC	16.95	12359	Cyprus
17	C580165	22720	SET OF 3 CAKE TINS PANTRY D...	-1	2011-12-02 11:21:00 UTC	4.95	12359	Cyprus
18	C544902	22273	FELTCRAFT DOLL MOLLY	-1	2011-02-24 13:05:00 UTC	2.95	12362	Belgium
19	C544902	22629	SPACEBOY LUNCH BOX	-1	2011-02-24 13:05:00 UTC	1.95	12362	Belgium

Q. 취소된 거래 건들은 어떠한 특징이나 경향성이 있나요?

Quantity가 -(음수)임

CustomerID로 분류 가능해 보임

StockCode, Description, UnitPrice, Country에서의 공통점 발견되지 않음

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT
  ROUND(
    SUM(
      CASE
        WHEN InvoiceNo LIKE 'C%' THEN 1  -- InvoiceNo가 'C'로 시작하면 1 = 취소로 카운트
        ELSE 0                           -- 그렇지 않으면 0
      END
    )
    / COUNT(*) * 100, 1   -- 전체 행 수(*) 대비 취소 거래 비율 계산, 소수점 첫째 자리까지 반올림
  ) AS cancelled_percentage -- 결과 컬럼 이름을 cancelled_percentage로 저장
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	cancelled_percentage
1	2.2

Q. 구매 건 상태가 `Canceled` 인 데이터의 비율을 소수점 첫 번째 자리까지 적어주세요.

2.2%

`StockCode` 살펴보기

- **고유한 `StockCode` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT StockCode) AS StockCode_count
FROM sonic-proxy-473602-k2.project01.data;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	StockCode_count
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM sonic-proxy-473602-k2.project01.data
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

Q. `StockCode` 별 등장 빈도를 출력했을 때, 일정한 경향성이 보이나요? 이상치는 있었나요?

StockCode는 보통 5~6자리 숫자로 이루어져 있는데, 간혹 숫자와 알파벳 문자 조합으로 구성되어 있거나, 알파벳 문자만 있는 경우도 있음(예. 85123A, POST)

✅ **현재 진행하고 있는 프로젝트는 고객들의 제품 구매에 초점이 맞춰져 있기 때문에 이런 값들은 제거하는 것이 좋을 것 같습니다.**

- **`StockCode` 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
-- 1. 고유한 StockCode만 추출하는 서브쿼리
WITH UniqueStockCodes AS (
  SELECT DISTINCT StockCode -- 중복 없이 StockCode만 선택
  FROM sonic-proxy-473602-k2.project01.data
)

-- 2. StockCode에서 숫자가 몇 개 포함되어 있는지 계산하고 그룹별 개수 확인
SELECT
  LENGTH(StockCode) -- StockCode 전체 길이
    - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count,
    -- REGEXP_REPLACE는 텍스트를 처리하는 정규 표현식(Regular Expressio)
    -- StockCode 값 중, 0~9를 제거한 뒤 ' ' 비어있는 값으로 처리
    -- 이후에 LENGTH 함수로 길이를 빼서 숫자 개수가 몇 개인지 계산
  COUNT(*) AS stock_cnt -- 숫자 개수가 동일한 StockCode 개수 카운트
FROM UniqueStockCodes
GROUP BY number_count   -- 숫자 개수별로 그룹화
ORDER BY stock_cnt DESC; -- 숫자 개수가 동일한 그룹 중 많은 순서대로 정렬
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	number_count	stock_cnt
1	5	3676
2	0	7
3	1	1

→ 숫자 5개로 이루어진 것 : 3676

→ 숫자 0개로 이루어진 것 : 7개

→ 숫자 1개로 이루어진 것 : 1개

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
-- 중복 제거 후 StockCode와 숫자 개수 확인
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM sonic-proxy-473602-k2.project01.data
)
-- 숫자가 0개 또는 1개인 StockCode만 선택
WHERE number_count BETWEEN 0 AND 1;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	StockCode	number_count
1	BANK CHARGES	0
2	C2	1
3	CRUK	0
4	D	0
5	DOT	0
6	M	0
7	PADS	0
8	POST	0

Q. 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있나요?

StockCode
C2
BANK CHARGES
CRUK
D
DOT
M
PADS
POST

- **해당 코드 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트일까요? 소수점 두번째 자리까지 구해 주세요.**

```
SELECT
      -- 소수점 둘째 자리까지 반올림 ROUND (, 2)
   ROUND(
         -- StockCode에 해당하는 행 수 계산 / 전체 데이터 행 수로 나누어 비율 계산
      COUNT(*) * 100.0 / (SELECT COUNT(*) FROM sonic-proxy-473602-k2.project01.data),
      2
   ) AS stockcode_percentage
FROM sonic-proxy-473602-k2.project01.data
WHERE StockCode IN (
   'C2',
   'BANK CHARGES',
   'CRUK',
   'D',
   'DOT',
   'M',
   'PADS',
   'POST'
);
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	stockcode_percentage
1	0.48

→ StockCode에 숫자를 0개 혹은 1개 가지고 있는 비율 : 0.48% (편향성 매우 적은 편)

⇒ 이 코드들은 'BANK CHARGES, POST' 등 제품과 관련되지 않은 거래 기록으로 보입니다. 진행하고 있는 프로젝트의 목표는 고객들의 '제품 구매'에 기반하여 세그먼테이션을 하는 것이므로, 이런 `StockCode`가 포함된 기록은 데이터셋에서 제외하도록 하겠습니다.

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
-- 삭제 전 정답과 동일한 행의 수가 삭제되는 것이 맞는지 확인
SELECT COUNT(*) AS to_be_deleted
FROM sonic-proxy-473602-k2.project01.data
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
          LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM sonic-proxy-473602-k2.project01.data
  )
  -- 숫자가 0개 또는 1개인 StockCode만 선택
  WHERE number_count BETWEEN 0 AND 1
);

-- 1,915행으로 정답과 동일한 점 확인 완료!
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	to_be_deleted
1	1915

```
-- 실제 표에서 삭제
DELETE FROM sonic-proxy-473602-k2.project01.data
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
          LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM sonic-proxy-473602-k2.project01.data
  )
  WHERE number_count BETWEEN 0 AND 1
);
```

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 data의 행 1,915개가 삭제되었습니다. 테이블로 이동

Description 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM sonic-proxy-473602-k2.project01.data
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	Description	description_cnt
1	WHITE HANGING HEART T-LIGHT HOLDER	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORNAMENT	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY DESIGN	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE CASES	1062
10	SPOTTY BUNTING	1026
11	PAPER CHAIN KIT 50'S CHRISTMAS	1013
12	LUNCH BAG SPACEBOY DESIGN	1006
13	LUNCH BAG CARS BLUE	1000
14	HEART OF WICKER SMALL	990
15	NATURAL SLATE HEART CHALKBOARD	989
16	JAM MAKING SET WITH JARS	966
17	LUNCH BAG PINK POLKADOT	961
18	LUNCH BAG SUKI DESIGN	932

→ 장식품, 주방용품, 도시락 용품 등이 많음

> ✅ **흥미롭게도 모든 Description이 대문자로 되어 있는데, 이는 데이터베이스에 제품 설명을 입력할 때 사용되는 표준화된 형식일 수도 있습니다. 그러나 혹시라도 대문자와 소문자가 혼합된 스타일로 입력된 설명이 있는지 확인해 보는 것이 현명할 것입니다.**

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE FROM sonic-proxy-473602-k2.project01.data
WHERE
   Description IN (
   'Next Day Carriage',
   'High Resolution Image'
);
```

MainProject_01_CustomerSem... | 실행 | 쿼리 저장 | 다운로드 | 공유

```
DELETE FROM sonic-proxy-473602-k2.project01.data
WHERE
    Description IN (
    'Next Day Carriage',
    'High Resolution Image'
);
```

쿼리 완료됨

쿼리 결과 | 결과 저장 | 다음에서 열기

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 data의 행 83개가 삭제되었습니다. | 테이블로 이동

→ 'Next Day Carriage', 'High Resolution Image'를 포함하는 총 83개의 행 삭제됨

- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
SELECT DISTINCT Description
FROM sonic-proxy-473602-k2.project01.data
WHERE REGEXP_CONTAINS(Description, r'[a-z]');
```

  - `REGEXP_CONTAINS`
    - **특정 패턴이 문자열에 포함되어 있는지 확인하는데 유용한 정규식 표현 함수**
    - **특정 패턴이 문자열에 포함되어 있으면 : True / 그렇지 않으면 : False**
    - `REGEXP_CONTAINS(Description, r'[a-z]')`

      → Description 문자열 중,
      - 소문자 알파벳 `[a-z]` 포함여부 확인
      - 만약 `r'[a-z]'` 대신 `r'[A-Z]'` 를 사용했다면 대문자 알파벳이 포함되어 있는지를 확인하는 조건문

→ 19개의 Description이 대문소 혼합 형태임

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

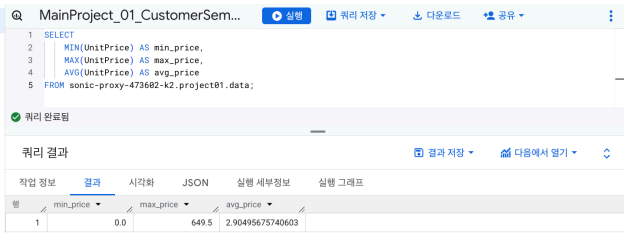
행	Description
1	BAG 250g SWIRLY MARBLES
2	3 TRADITIONAl BISCUIT CUTTERS SET
3	BAG 125g SWIRLY MARBLES
4	POLYESTER FILLER PAD 30CMx30CM
5	BAG 500g SWIRLY MARBLES
6	POLYESTER FILLER PAD 45x45cm
7	POLYESTER FILLER PAD 40x40cm
8	ESSENTIAL BALM 3.5g TIN IN ENVELOPE
9	High Resolution Image
10	FRENCH BLUE METAL DOOR SIGN No
11	POLYESTER FILLER PAD 45x30cm
12	Next Day Carriage
13	NUMBER TILE VINTAGE FONT No
14	NUMBER TILE COTTAGE GARDEN No
15	POLYESTER FILLER PAD 65CMx65CM
16	FLOWERS HANDBAG blue and orange
17	FOLK ART GREETING CARD,pack/12
18	THE KING GIFT BAG 25x24x12cm
19	POLYESTER FILLER PAD 60x40cm

> Description 소문자로 작성된 19개 처리 전략
> 1) **'Next Day Carriage'와 'High Resolution Image'와 같은 서비스 관련 정보를 포함하는 행들을 제거**
> **2) 대소문자를 혼합해서 사용하는 경우, 대문자로 표준화하여 데이터셋 전체에서 일관성을 유지**

UnitPrice 살펴보기

- **UnitPrice 의 최솟값, 최댓값, 평균을 구하기**

```
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM sonic-proxy-473602-k2.project01.data;
```

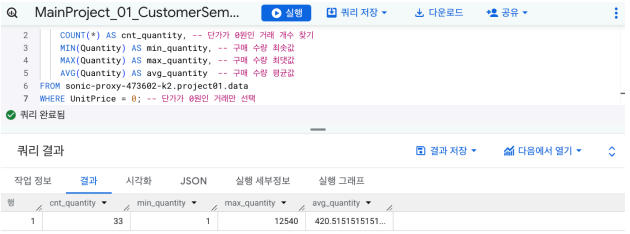


Q. UnitPrice 데이터에 특이한 값이 있나요?

개당 단가가 0원인 데이터가 존재
→ 제품이 무료?!
→ 오타?! 오류?!

- **단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기**

```
SELECT
   COUNT(*) AS cnt_quantity, -- 단가가 0원인 거래 개수 찾기
   MIN(Quantity) AS min_quantity, -- 구매 수량 최솟값
   MAX(Quantity) AS max_quantity, -- 구매 수량 최댓값
   AVG(Quantity) AS avg_quantity  -- 구매 수량 평균값
FROM sonic-proxy-473602-k2.project01.data
WHERE UnitPrice = 0; -- 단가가 0원인 거래만 선택
```



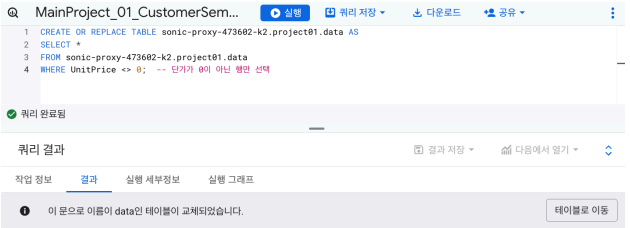
→ 구매 개수 : 최소 1개 ~ 최대 12,540개

→ 구매 개수 = 0인 행의 수는 33개

→ 데이터의 수가 적은 걸 보니 무료 제품이라기보다 데이터 오류일 가능성이 더 높을 것 같습니다.

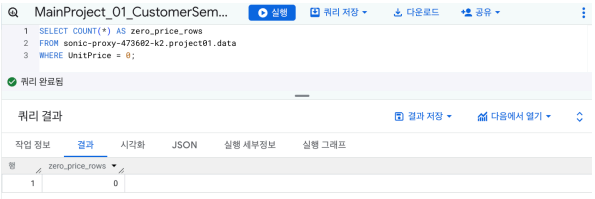
- `UnitPrice = 0` **를 제거하고 일관된 데이터셋을 유지하기**

```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.data AS
SELECT *
FROM sonic-proxy-473602-k2.project01.data
WHERE UnitPrice <> 0;  -- 단가가 0이 아닌 행만 선택
```

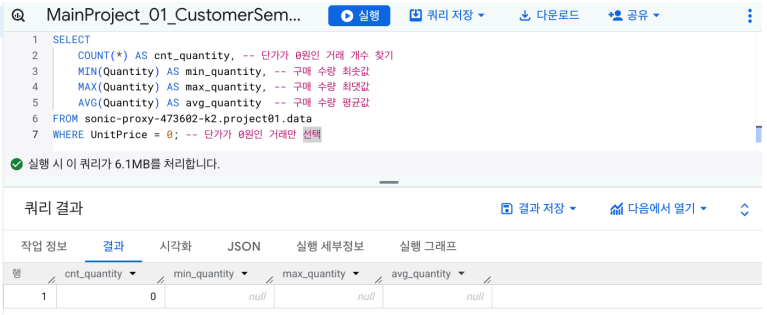


UnitPrice = 0인 행이 모두 지워졌는지 다시 체크

```
SELECT COUNT(*) AS zero_price_rows
FROM sonic-proxy-473602-k2.project01.data
WHERE UnitPrice = 0;

-- 확인 완료!!
```




```
SELECT
   COUNT(*) AS cnt_quantity, -- 단가가 0원인 거래 개수 찾기
   MIN(Quantity) AS min_quantity, -- 구매 수량 최솟값
   MAX(Quantity) AS max_quantity, -- 구매 수량 최댓값
   AVG(Quantity) AS avg_quantity  -- 구매 수량 평균값
FROM sonic-proxy-473602-k2.project01.data
WHERE UnitPrice = 0; -- 단가가 0원인 거래만 선택

-- 확인 완료!!
```

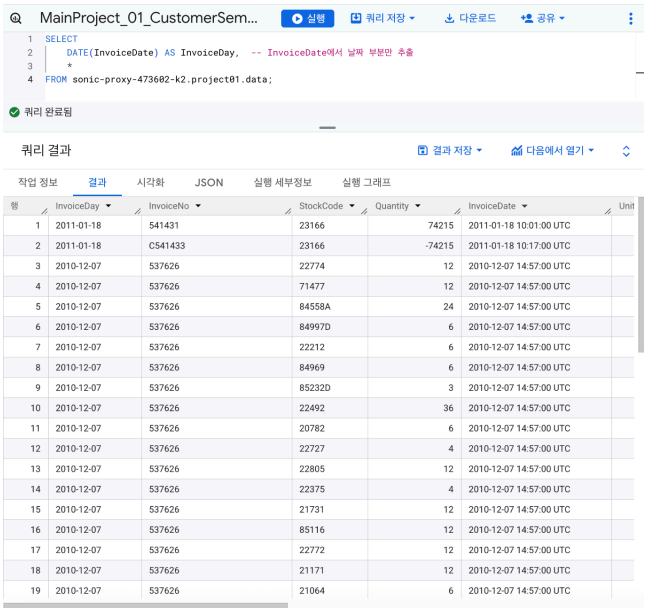


11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT
   DATE(InvoiceDate) AS InvoiceDay,  -- InvoiceDate에서 날짜 부분만 추출
   *
FROM sonic-proxy-473602-k2.project01.data;
```



- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT
      -- 데이터 전체 기준 가장 최근 구매일
      -- 각 InvoiceDay 행에 동일하게 최근 구매일을 입력하기 위해 OVER () 추가
   MAX(DATE(InvoiceDate)) OVER () AS most_recent_date,
   -- InvoiceDate에서 날짜만 추출
   DATE(InvoiceDate) AS InvoiceDay,
   *
FROM sonic-proxy-473602-k2.project01.data;

-- 정답 : 2011-12-09
```

MainProject_01_CustomerSem... 실행 일정 다음에서 열기 더보기 쿼리 저장 다운로드 공유

```
SELECT
    -- 데이터 전체 기준 가장 최근 구매일
    MAX(DATE(InvoiceDate)) OVER () AS most_recent_date,
    -- InvoiceDate에서 날짜만 추출
    DATE(InvoiceDate) AS InvoiceDay,
    *
FROM sonic-proxy-473602-k2.project01.data;
```

쿼리 완료됨

쿼리 결과 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-12-09	2011-11-18	577228	21528	1	2011-11-18 12:07:00 UTC	6.95	12391	Cyprus	DAIRY MAID TRADITIONAL TEA...
2	2011-12-09	2011-03-03	545475	22846	16	2011-03-03 10:59:00 UTC	14.95	12415	Australia	BREAD BIN DINER STYLE RED
3	2011-12-09	2011-03-03	545475	22978	72	2011-03-03 10:59:00 UTC	3.39	12415	Australia	PANTRY ROLLING PIN
4	2011-12-09	2011-06-15	556917	21080	96	2011-06-15 13:37:00 UTC	0.72	12415	Australia	SET/20 RED RETROSPOT PAPE...
5	2011-12-09	2011-03-21	547109	23008	1	2011-03-21 11:46:00 UTC	16.95	12423	Belgium	DOLLY GIRL BABY GIFT SET
6	2011-12-09	2011-10-14	571255	22173	32	2011-10-14 17:13:00 UTC	2.75	12454	Spain	METAL 4 HOOK HANGER FREN...
7	2011-12-09	2011-08-25	564481	22847	4	2011-08-25 13:33:00 UTC	14.95	12464	Belgium	BREAD BIN DINER STYLE IVORY
8	2011-12-09	2011-10-25	572559	23318	12	2011-10-25 08:44:00 UTC	2.49	12476	Germany	BOX OF 6 MINI VINTAGE CRACK...
9	2011-12-09	2011-10-14	571227	84792	8	2011-10-14 13:54:00 UTC	4.65	12477	Germany	ENCHANTED BIRD COATHANGE...
10	2011-12-09	2011-01-07	540469	47590A	4	2011-01-07 14:04:00 UTC	5.45	12484	Spain	BLUE HAPPY BIRTHDAY BUNTI...
11	2011-12-09	2011-11-01	573868	21055	3	2011-11-01 12:18:00 UTC	8.95	12491	France	TOOL BOX SOFT TOY
12	2011-12-09	2011-01-07	540458	21882	24	2011-01-07 12:28:00 UTC	0.19	12501	Germany	SKULLS TAPE
13	2011-12-09	2011-10-11	570672	22232	16	2011-10-11 14:52:00 UTC	0.59	12536	France	JIGSAW TOADSTOOLS 3 PIECE
14	2011-12-09	2011-01-27	542303	22509	1	2011-01-27 11:05:00 UTC	16.95	12539	Spain	SEWING BOX RETROSPOT DESI...
15	2011-12-09	2011-12-01	579927	23406	2	2011-12-01 09:20:00 UTC	6.25	12572	Germany	HOME SWEET HOME KEY HOLD...
16	2011-12-09	2011-11-22	577809	22844	12	2011-11-22 08:34:00 UTC	7.65	12584	Italy	VINTAGE CREAM DOG FOOD C...

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay -- 고객별 가장 최근 구매일
FROM sonic-proxy-473602-k2.project01.data
GROUP BY CustomerID; -- CustomerID별로 그룹화
```

MainProject_01_CustomerSem... 실행 쿼리 저장 다운로드 공유

```
SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay -- 고객별 가장 최근 구매일
FROM sonic-proxy-473602-k2.project01.data
GROUP BY CustomerID; -- CustomerID별로 그룹화
```

실행 시 이 쿼리가 6.1MB를 처리합니다.

쿼리 결과 결과 저장 다음에서 열기

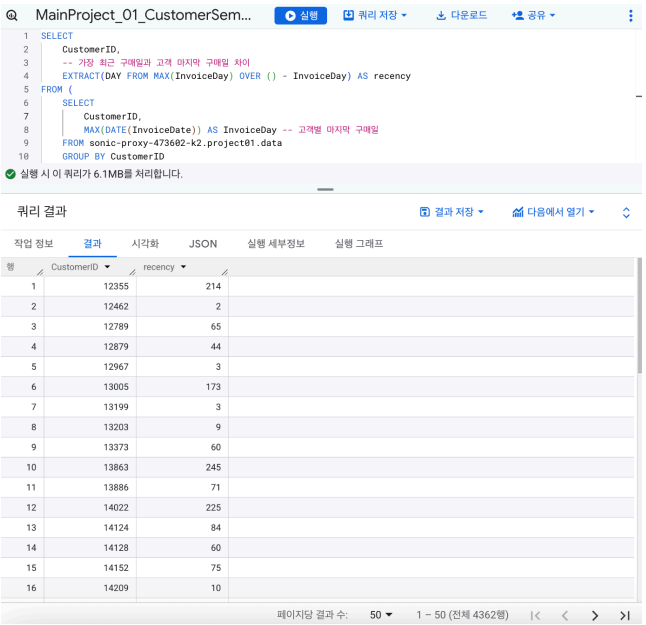
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17
11	12357	2011-11-06
12	12358	2011-12-08
13	12359	2011-12-02
14	12360	2011-10-18
15	12361	2011-02-25
16	12362	2011-12-06

- **가장 최근 일자(most_recent_date)와 유저별 마지막 구매일(InvoiceDay)간의 차이를 계산하기**

```
SELECT
    CustomerID,
    -- 가장 최근 구매일과 고객 마지막 구매일 차이
    EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
```

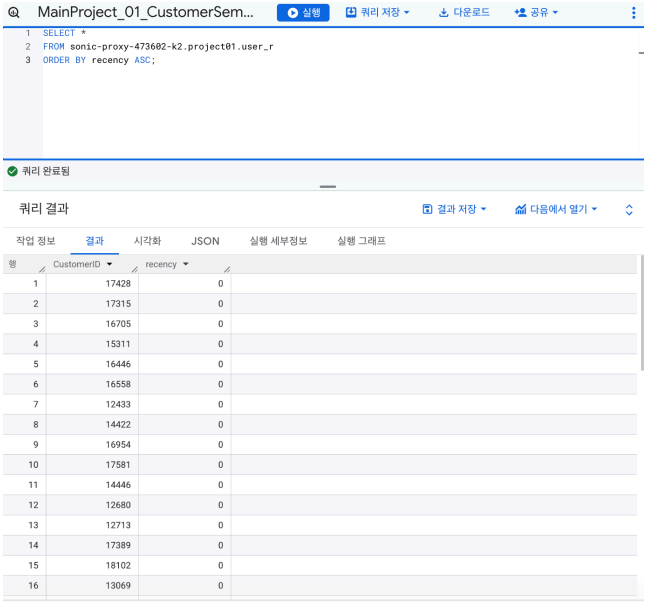
```
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay -- 고객별 마지막 구매일
  FROM sonic-proxy-473602-k2.project01.data
  GROUP BY CustomerID
);
```



- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.user_r AS
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency  -- Recency 계산
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM sonic-proxy-473602-k2.project01.data
  GROUP BY CustomerID
);
```

```
SELECT *
FROM sonic-proxy-473602-k2.project01.user_r
ORDER BY recency ASC;
```

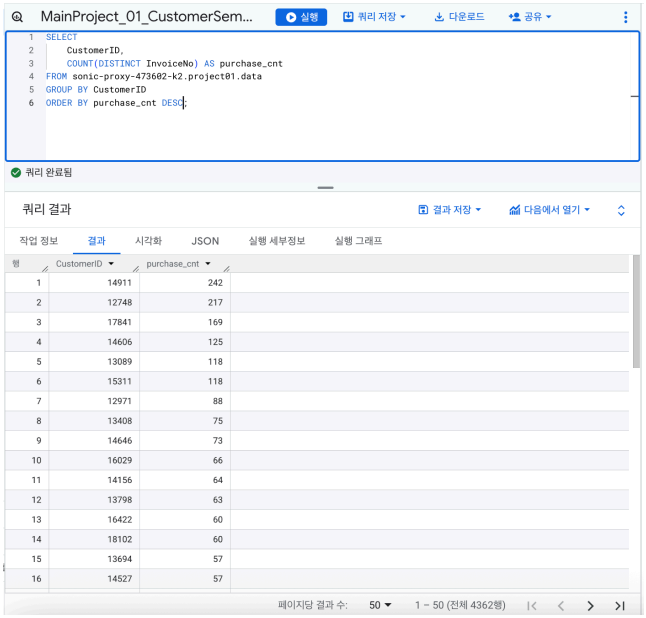


행	CustomerID	recency
1	17428	0
2	17315	0
3	16705	0
4	15311	0
5	16446	0
6	16558	0
7	12433	0
8	14422	0
9	16954	0
10	17581	0
11	14446	0
12	12680	0
13	12713	0
14	17389	0
15	18102	0
16	13069	0

Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM sonic-proxy-473602-k2.project01.data
GROUP BY CustomerID
ORDER BY purchase_cnt DESC;
```

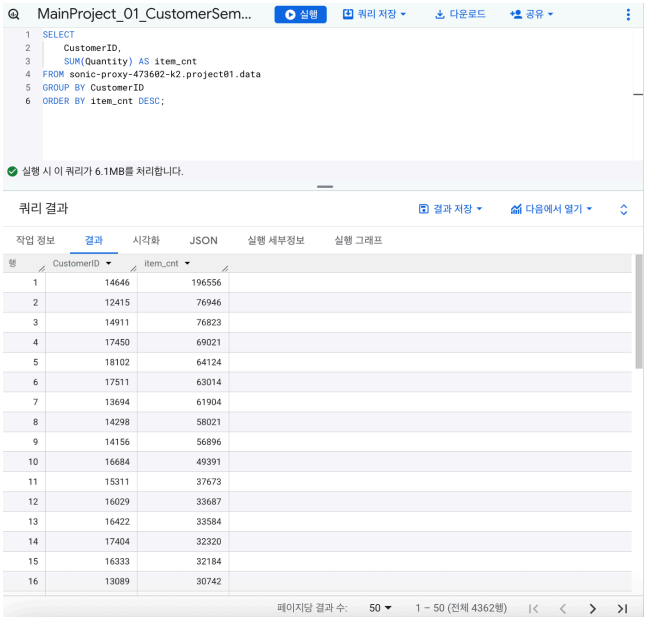


행	CustomerID	purchase_cnt
1	14911	242
2	12748	217
3	17841	169
4	14606	125
5	13089	118
6	15311	118
7	12971	88
8	13408	75
9	14646	73
10	16029	66
11	14156	64
12	13798	63
13	16422	60
14	18102	60
15	13694	57
16	14527	57

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

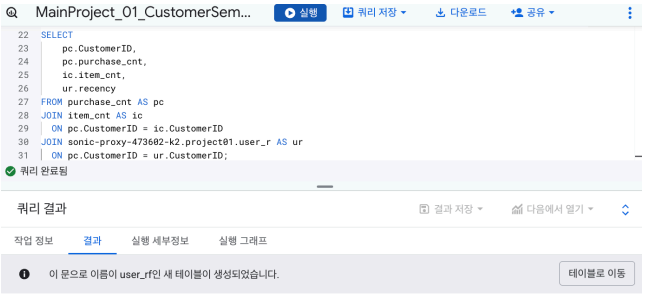
```
SELECT
    CustomerID,
```

```
    SUM(Quantity) AS item_cnt
FROM sonic-proxy-473602-k2.project01.data
GROUP BY CustomerID
ORDER BY item_cnt DESC;
```



- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

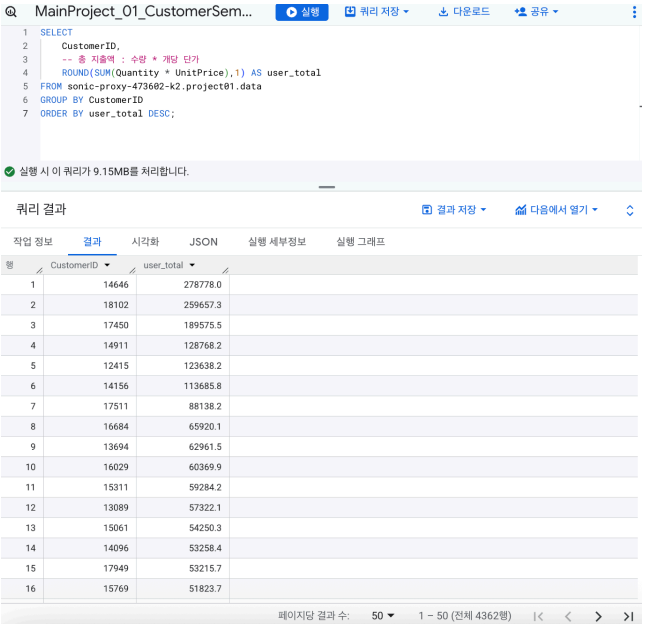
```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.user_rf AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  SELECT
      CustomerID,
      COUNT(DISTINCT InvoiceNo) AS purchase_cnt -- 고객별 고유 거래 건수
  FROM sonic-proxy-473602-k2.project01.data
  GROUP BY CustomerID
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
  SELECT
      CustomerID,
      SUM(Quantity) AS item_cnt -- 고객별 총 구매 수량
  FROM sonic-proxy-473602-k2.project01.data
  GROUP BY CustomerID
)

-- 기존 user_r 테이블과 통합
SELECT
    pc.CustomerID,
    pc.purchase_cnt,
    ic.item_cnt,
    ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN sonic-proxy-473602-k2.project01.user_r AS ur
  ON pc.CustomerID = ur.CustomerID;
```



Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
    CustomerID,
    -- 총 지출액 : 수량 * 개당 단가
    ROUND(SUM(Quantity * UnitPrice),1) AS user_total
FROM sonic-proxy-473602-k2.project01.data
GROUP BY CustomerID
ORDER BY user_total DESC;
```



- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt`로 나누어서 3) `user_rfm` 테이블로 저장하기**

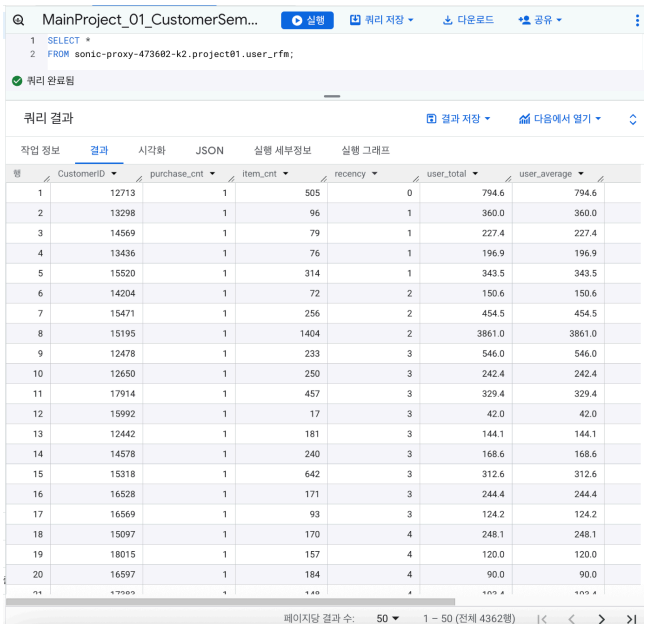
```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.user_rfm AS
SELECT
    rf.CustomerID AS CustomerID,
    rf.purchase_cnt,
    rf.item_cnt,
    rf.recency,
    -- 1차 결과값 확인 후, '부동소수점 연산'으로 소수점 0이 길게 표시되어
```

```
    -- 이를 보기 좋게 바꾸기 위해 user_total에도 소수점 첫째자리 반올림 적용
    ROUND(ut.user_total, 1) AS user_total,
    -- 거래당 평균 거래 금액 (총금액/거래횟수)
    ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_average
FROM sonic-proxy-473602-k2.project01.user_rf rf
LEFT JOIN (
    -- 고객별 총 거래액 계산
    SELECT
        CustomerID,
        SUM(Quantity * UnitPrice) AS user_total
    FROM sonic-proxy-473602-k2.project01.data
    GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```

RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM sonic-proxy-473602-k2.project01.user_rfm;
```

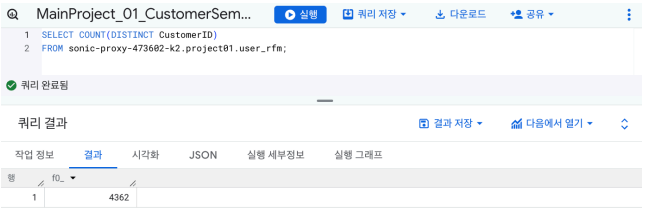


행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.6	794.6
2	13298	1	96	1	360.0	360.0
3	14569	1	79	1	227.4	227.4
4	13436	1	76	1	196.9	196.9
5	15520	1	314	1	343.5	343.5
6	14204	1	72	2	150.6	150.6
7	15471	1	256	2	454.5	454.5
8	15195	1	1404	2	3861.0	3861.0
9	12478	1	233	3	546.0	546.0
10	12650	1	250	3	242.4	242.4
11	17914	1	457	3	329.4	329.4
12	15992	1	17	3	42.0	42.0
13	12442	1	181	3	144.1	144.1
14	14578	1	240	3	168.6	168.6
15	15318	1	642	3	312.6	312.6
16	16528	1	171	3	244.4	244.4
17	16569	1	93	3	124.2	124.2
18	15097	1	170	4	248.1	248.1
19	18015	1	157	4	120.0	120.0
20	16597	1	184	4	90.0	90.0

  - 전체 4,362행으로 행의 개수 동일한 점 확인!
  - 컬럼 값도 정답과 동일한 점 확인!
  - **Q. 고유한 유저의 수는 몇 명인가요?**

4,362명

```
SELECT COUNT(DISTINCT CustomerID)
FROM sonic-proxy-473602-k2.project01.user_rfm;
```

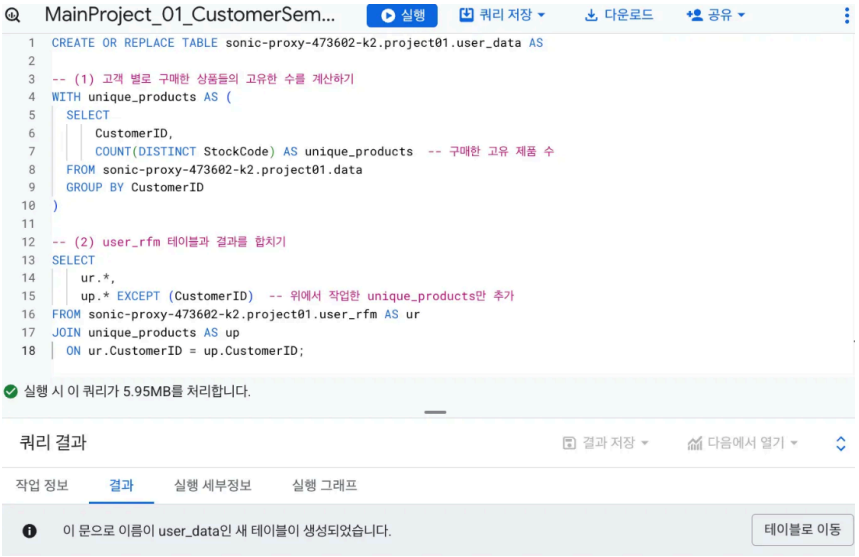


11-8. 추가 Feature 추출

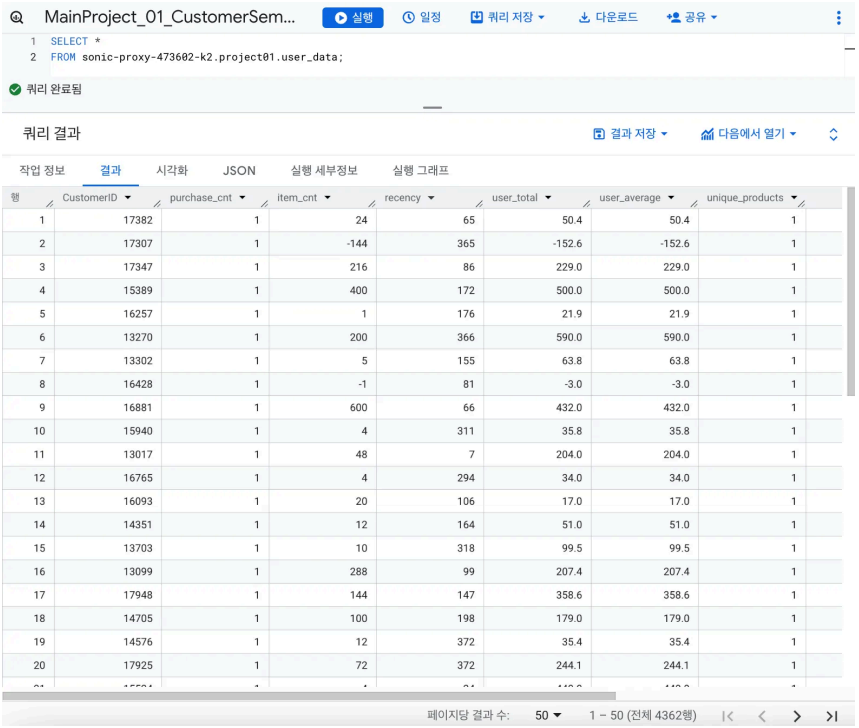
1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.user_data AS

-- (1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
WITH unique_products AS (
  SELECT
      CustomerID,
      COUNT(DISTINCT StockCode) AS unique_products  -- 구매한 고유 제품 수
  FROM sonic-proxy-473602-k2.project01.data
  GROUP BY CustomerID
)

-- (2) user_rfm 테이블과 결과를 합치기
SELECT
    ur.*,
    up.* EXCEPT (CustomerID)  -- 위에서 작업한 unique_products만 추가
FROM sonic-proxy-473602-k2.project01.user_rfm AS ur
JOIN unique_products AS up
  ON ur.CustomerID = up.CustomerID;
```



```
SELECT *
FROM sonic-proxy-473602-k2.project01.user_data;
```



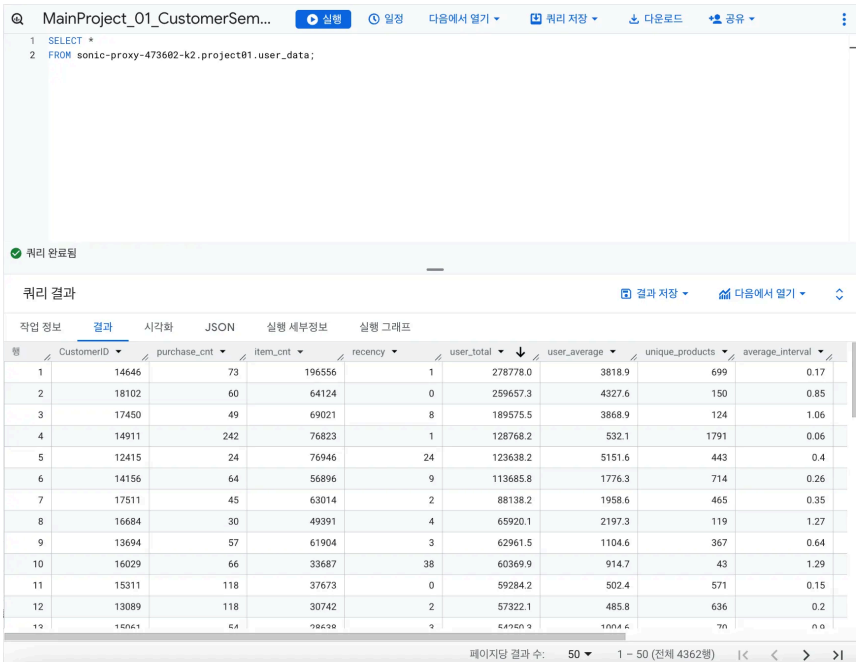
행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products
1	17382	1	24	65	50.4	50.4	1
2	17307	1	-144	365	-152.6	-152.6	1
3	17347	1	216	86	229.0	229.0	1
4	15389	1	400	172	500.0	500.0	1
5	16257	1	1	176	21.9	21.9	1
6	13270	1	200	366	590.0	590.0	1
7	13302	1	5	155	63.8	63.8	1
8	16428	1	-1	81	-3.0	-3.0	1
9	16881	1	600	66	432.0	432.0	1
10	15940	1	4	311	35.8	35.8	1
11	13017	1	48	7	204.0	204.0	1
12	16765	1	4	294	34.0	34.0	1
13	16093	1	20	106	17.0	17.0	1
14	14351	1	12	164	51.0	51.0	1
15	13703	1	10	318	99.5	99.5	1
16	13099	1	288	99	207.4	207.4	1
17	17948	1	144	147	358.6	358.6	1
18	14705	1	100	198	179.0	179.0	1
19	14576	1	12	372	35.4	35.4	1
20	17925	1	72	372	244.1	244.1	1

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.user_data AS

-- 1. 고객별 평균 구매 소요 일수 계산
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    -- 평균 구매 텀 계산, NULL = 0, 소수점 둘째 자리 반올림
    CASE
      WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0
      ELSE ROUND(AVG(interval_), 2)
    END AS average_interval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(DATE(InvoiceDate), LAG(DATE(InvoiceDate)) OVER (PARTITION BY CustomerID ORDER BY DATE(InvoiceDate)),
DAY) AS interval_
    FROM sonic-proxy-473602-k2.project01.data
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)
```

```
-- 2. 기존 user_data 테이블에 통합
SELECT
   u.*,
   pi.* EXCEPT (CustomerID)  -- average_interval만 추가
FROM sonic-proxy-473602-k2.project01.user_data AS u
LEFT JOIN purchase_intervals AS pi
  ON u.CustomerID = pi.CustomerID;
```



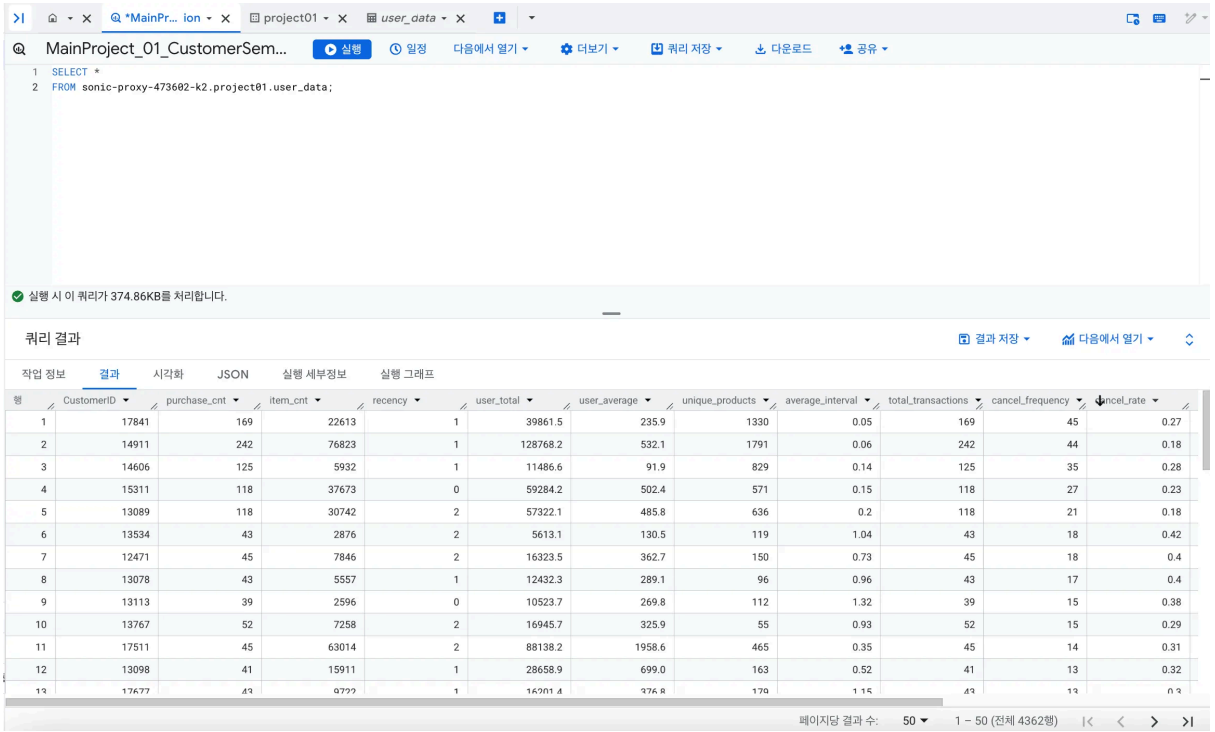
행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval
1	14646	73	196556	1	278778.0	3818.9	699	0.17
2	18102	60	64124	0	259657.3	4327.6	150	0.85
3	17450	49	69021	8	189575.5	3868.9	124	1.06
4	14911	242	76823	1	128768.2	532.1	1791	0.06
5	12415	24	76946	24	123638.2	5151.6	443	0.4
6	14156	64	56896	9	113685.8	1776.3	714	0.26
7	17511	45	63014	2	88138.2	1958.6	465	0.35
8	16684	30	49391	4	65920.1	2197.3	119	1.27
9	13694	57	61904	3	62961.5	1104.6	367	0.64
10	16029	66	33687	38	60369.9	914.7	43	1.29
11	15311	118	37673	0	59284.2	502.4	571	0.15
12	13089	118	30742	2	57322.1	485.8	636	0.2

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE sonic-proxy-473602-k2.project01.user_data AS

-- (1) 고객별 전체 거래 건수와 취소 거래 건수 계산
WITH TransactionInfo AS (
  SELECT
     CustomerID,
     COUNT(DISTINCT InvoiceNo) AS total_transactions, -- 고객별 전체 거래 건수
     COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency -- 고객별 취소 거래 건수
  FROM sonic-proxy-473602-k2.project01.data
  GROUP BY CustomerID
)

-- (2) 기존 user_data 테이블과 결합
SELECT
   u.*,
   t.* EXCEPT(CustomerID),  -- cancel_frequency와 total_transactions만 추가
   -- 고객별 취소 비율 = 취소 거래 / 전체 거래
   ROUND(IF(t.total_transactions = 0, 0, t.cancel_frequency / t.total_transactions), 2) AS cancel_rate
FROM sonic-proxy-473602-k2.project01.user_data AS u
```

```
LEFT JOIN TransactionInfo AS t
  ON u.CustomerID = t.CustomerID;
```



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	17841	169	22613	1	39861.5	235.9	1330	0.05	169	45	0.27
2	14911	242	76823	1	128768.2	532.1	1791	0.06	242	44	0.18
3	14606	125	5932	1	11486.6	91.9	829	0.14	125	35	0.28
4	15311	118	37673	0	59284.2	502.4	571	0.15	118	27	0.23
5	13089	118	30742	2	57322.1	485.8	636	0.2	118	21	0.18
6	13534	43	2876	2	5613.1	130.5	119	1.04	43	18	0.42
7	12471	45	7846	2	16323.5	362.7	150	0.73	45	18	0.4
8	13078	43	5557	1	12432.3	289.1	96	0.96	43	17	0.4
9	13113	39	2596	0	10523.7	269.8	112	1.32	39	15	0.38
10	13767	52	7258	2	16945.7	325.9	55	0.93	52	15	0.29
11	17511	45	63014	2	88138.2	1958.6	465	0.35	45	14	0.31
12	13098	41	15911	1	28658.9	699.0	163	0.52	41	13	0.32
13	17677	43	9722	1	16201.4	376.8	179	1.15	43	13	0.3

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM sonic-proxy-473602-k2.project01.user_data
ORDER BY
    recency ASC,
    purchase_cnt DESC,
    tem_cnt DESC;
```

MainProject_01_CustomerSem... 실행 일정 다음에서 열기 더보기 쿼리 저장 다운로드 공유

```
-- 최종 user_data 테이블 확인
SELECT *
FROM sonic-proxy-473602-k2.project01.user_data
ORDER BY recency ASC, purchase_cnt DESC, item_cnt DESC;
```

쿼리 완료됨

쿼리 결과 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	12748	217	23516	0	29820.0	137.4	1767	0.08	217	11	0.05
2	15311	118	37673	0	59284.2	502.4	571	0.15	118	27	0.23
3	18102	60	64124	0	259657.3	4327.6	150	0.85	60	0	0.0
4	17389	42	7442	0	31317.5	745.7	47	1.53	42	8	0.19
5	17315	41	3805	0	6153.3	150.1	257	0.74	41	5	0.12
6	13777	40	12807	0	25758.1	644.0	39	1.73	40	7	0.17
7	13113	39	2596	0	10523.7	269.8	112	1.32	39	15	0.38
8	17428	33	9435	0	17078.5	517.5	255	1.08	33	5	0.15
9	17581	31	5849	0	10716.3	345.7	231	0.83	31	6	0.19
10	16705	29	5458	0	13946.1	480.9	135	1.27	29	9	0.31
11	13069	27	5454	0	3713.1	137.5	285	0.79	27	3	0.11
12	14051	26	3740	0	15462.3	594.7	51	1.72	26	3	0.12
13	16558	24	5346	0	8257.0	344.0	264	0.77	24	4	0.17

페이지당 결과 수: 50 1 – 50 (전체 4362행)

회고

Keep :

- 처음 진행하는 프로젝트로 이해하기 어려운 내용이었지만, 노드 내 참고할 수 있는 힌트 쿼리문을 기반으로 단계별 문제를 스스로 해결해보고자 노력함
- 평균 구매 주기, 취소 빈도와 취소 비율까지 계산하면서 쿼리 연습 문제 형식으로 학습하기 좋았음
- 기존에 GA나 AA를 통해 조회했던 RFM의 백엔드 데이터 구조와 쿼리 실행 경험을 직접해 볼 수 있어서 뜻깊은 시간이었음

Problem :

- 기초 SQL은 어느 정도 실사용이 가능하지만,
  서브쿼리 작성, WITH 문, SELECT 절 내 조건 구성은 아직 많이 어렵게 느껴짐
- 컬럼 중복 해결을 위해 EXCEPT나 날짜/시간 등을 효율적으로 처리하기 위한 정규식 표현(REGEXP_REPLACE)을 새롭게 배워야 했고, 익히는데 시간이 소요되었음
- 소수점 계산에서 부동소수점이 발견되어 나누기 연산 시, ROUND()를 추가로 적용해야 결과가 깔끔하게 보여진다는 것을 배움

Try :

- 서브쿼리, JOIN, WITH 문 활용 연습을 통해 중급 수준의 쿼리 작성 능력 향상 필요
- 초기 데이터 구조를 충분히 숙지하고, 스스로 쿼리를 설계하고 작성하는 연습 지속 필요